UNIVERSITÀ DI TRENTO

Web and HTTP

Parzialmente tratto da Computer Networking: A Top-Down Approach, Kurose-Ross

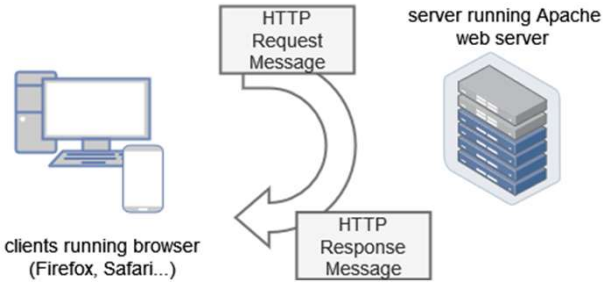
1

HTTP overview

"The Hypertext Transfer Protocol (HTTP) is an application-level protocol for distributed, collaborative, hypermedia information systems. It is a generic, stateless, protocol which can be used for many tasks beyond its use for hypertext, […], through extension of its request methods, error codes and headers." (RFC2616)

2

HTTP overview

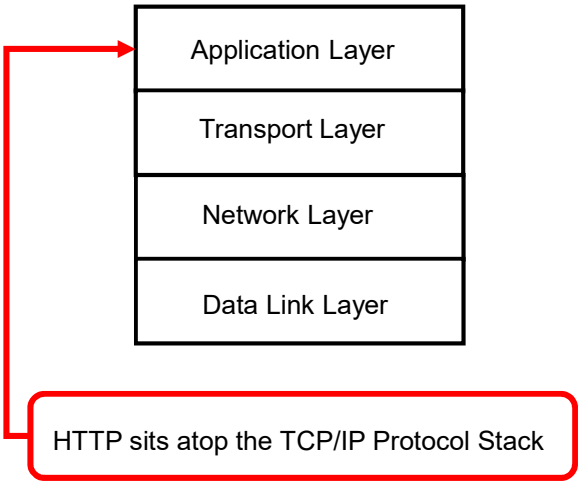


HTTP is an asymmetric request-response client-server protocol: a client sends a request message to a server returning a response message.

HTTP is a pull protocol, the client pulls information from the server (instead of server pushes information down to the client).

3

HTTP and TCP/IP



4

HTTP overview (continued)

HTTP uses TCP:

- a client initiates TCP connection (creates socket) to server, port 80
- the server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

5

HTTP overview (continued)

HTTP is stateless

- server maintains no information about past client requests

protocols that maintain “state” are complex:

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

6

HTTP overview (continued)

A protocol is stateful

- when the server maintains information about past client requests

e.g. FTP:

- you can issue a “cd” command to move into a (remote) directory
- The next commands (e.g. “ls”) will be executed with reference to that directory

7

Q

Where is HTTP defined ?

8

HTTP

- The Original HTTP as defined in 1991 as HTTP 0.9
- RFC 1945 HTTP/1.0 (1996)
- RFC 2616 HTTP/1.1 (1999) —redefined→ RFC 7230 HTTP/1.1 (2014)

HTTP/0.9	HTTP/1.0	HTTP/1.1
telnet-friendly protocol	Browser-friendly protocol	Browser-friendly protocol
No HTTP headers, error codes, versioning..	Header fields / metadata (HTTP version number, status code, content type)	Performance optimizations and feature enhancements (persistent and pipelined connections, virtual hosting...)
Methods supported: GET	Methods supported: GET, HEAD, POST	Methods supported: GET, HEAD, POST, PUT, DELETE, TRACE, OPTIONS
Response type: hypertext	Response: not limited to hypertext (Content-Type header enable transmission of files other than plain HTML files)	Response: as v1.0
Connection nature: terminated immediately after the response	Connection nature: terminated immediately after the response	Connection nature: long-lived

9

Something new

HTTP/2 and HTTP/3

RFC 7540 HTTP/2.0 (2015)

- Almost standard nowadays
- The way of working with web apps doesn't change; a reduction in page load latency is ensured
- HTTP/2 is used by 35% of all the websites (January 2026)

10

Web and HTTP

- A web page consists of objects
- …objects: text file, JPEG image, audio file,…

- A web page contains of base HTML-file which includes several referenced objects
- …each object is addressable by a URL:
  - www.somecompany.com/someDept/pic.gif
  - www. somecompany.com is the host name
  - someDept/pic.gif is the path to the object

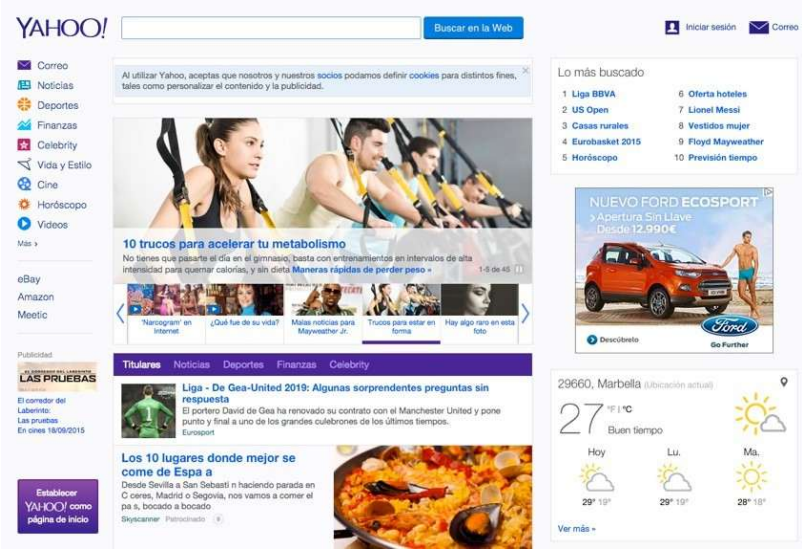
11

Web in 1996

- **Arts** - - *Humanities*, *Photography*, *Architecture*, *...*
- **Business and Economy [Xtra!]** - - *Directory*, *Investments*, *Classifieds*, *...*
- **Computers and Internet [Xtra!]** - - *Internet*, *WWW*, *Software*, *Multimedia*, *...*
- **Education** - - *Universities*, *K-12*, *Courses*, *...*
- **Entertainment [Xtra!]** - - *TV*, *Movies*, *Music*, *Magazines*, *...*
- **Government** - - *Politics [Xtra!]*, *Agencies*, *Law*, *Military*, *...*
- **Health [Xtra!]** - - *Medicine*, *Drugs*, *Diseases*, *Fitness*, *...*
- **News [Xtra!]** - - *World [Xtra!]*, *Daily*, *Current Events*, *...*
- **Recreation and Sports [Xtra!]** - - *Sports*, *Games*, *Travel*, *Autos*, *Outdoors*, *...*
- **Reference** - - *Libraries*, *Dictionaries*, *Phone Numbers*, *...*
- **Regional** - - *Countries*, *Regions*, *U.S. States*, *...*
- **Science** - - *CS*, *Biology*, *Astronomy*, *Engineering*, *...*
- **Social Science** - - *Anthropology*, *Sociology*, *Economics*, *...*
- **Society and Culture** - - *People*, *Environment*, *Religion*, *...*

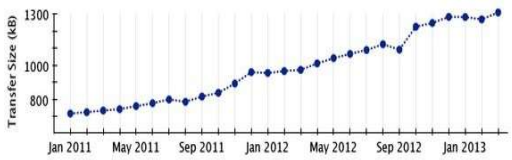
12

Web Today



13

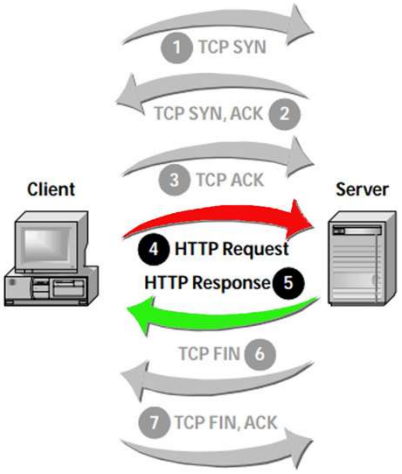
Our applications are complex, and growing...



Content Type	Desktop		Mobile	
	Avg # of requests	Avg size	Avg # of requests	Avg size
HTML	10	56 KB	6	40 KB
Images	56	856 KB	38	498 KB
Javascript	15	221 KB	10	146 KB
CSS	5	36 KB	3	27 KB
Total	86+	1169+ KB	57+	711+ KB

14

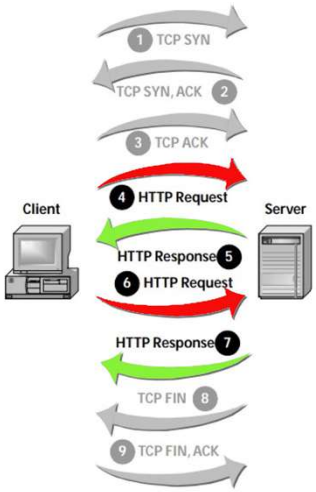
HTTP requires a TCP connection



Before systems can exchange HTTP messages, they must establish a TCP connection. Steps 1, 2, and 3 in this example show the connection establishment. Once the TCP connection is available, the client sends the server an HTTP request. The final two steps, 6 and 7, show the closing of the TCP connection.

15

Persistent connections

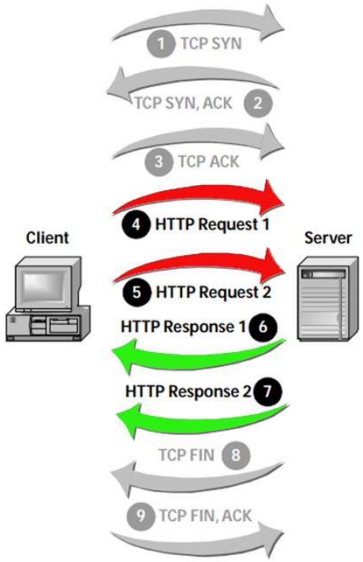


With persistent connections, a client can issue many HTTP requests over a single TCP connection. The first request is in step 4, which the server answers in step 5. In step 6 the client continues by sending the server another request on the same TCP connection. The server responds to this request in step 7 and then closes the TCP connection.

16

Pipelining

Pipelining lets an HTTP client issue new requests without waiting for responses from its previous messages. In the figure, the client sends its first request in step 4. It immediately follows that with a second request in step 5. The client does not wait for the server's response, which arrives in step 6.



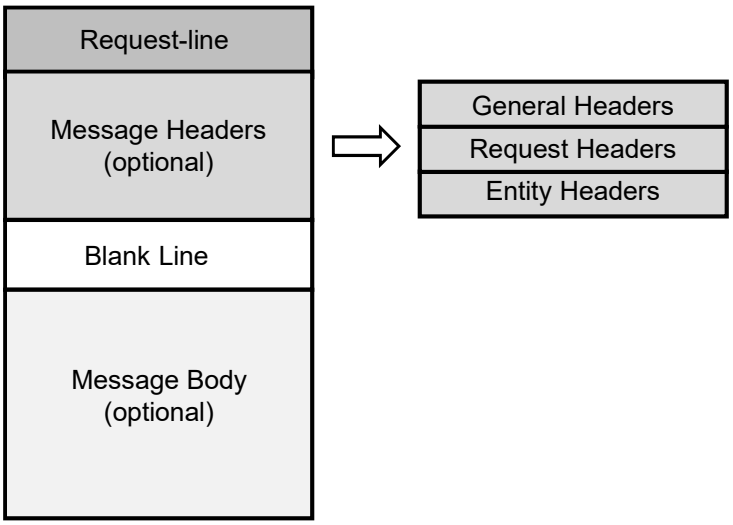
17

Q

How does the HTTP request composed ?

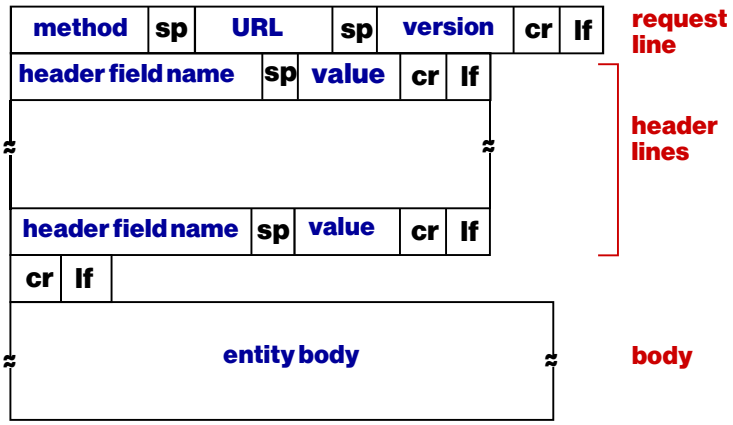
18

HTTP request format



19

Anatomy of an HTTP request message



20

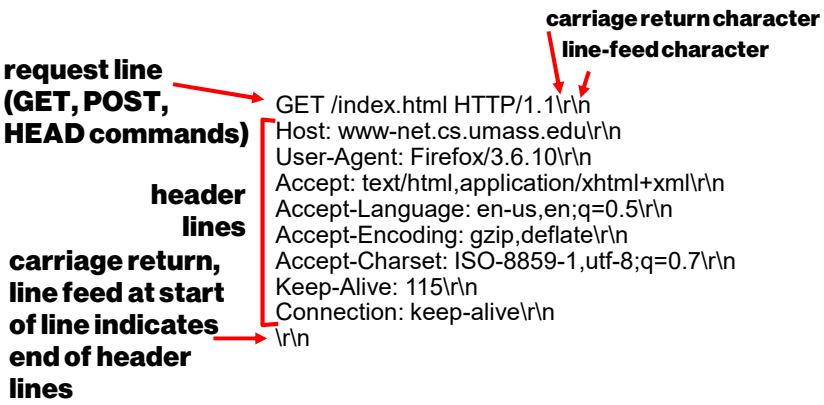
HTTP requests

- HTTP requests are a type of Internet Messages (RFC 822), and have the format format:
  - A Start Line, followed by a CRLF
    Request Line
  - Zero or more Message Headers
    Field-name ":" [field-value] CRLF
  - An empty line
    Two CRLFs mark the end of the Headers
  - An optional Message Body if there is a payload
    All or part of the "Entity Body" or "Entity"

21

An example of a HTTP request message

- HTTP request message:
  - ASCII (human-readable format)



22

Request Methods

HTTP/1.0

- **HEAD**
  - Retrieves only the Headers associated with a resource but not the entity itself
  - Highly useful for protocol analysis, diagnostics
- **GET**
  - Requests a resource from the server
  - Supports passing of query string arguments
- **POST**
  - Allows passing (submitting) of data in entity rather than URL
  - Appends resources to the existing ones
  - Can transmit of far larger arguments that GET
  - Arguments not displayed on the URL

23

Request Methods, cont.

HTTP/1.1 adds:

- **OPTIONS**
  - Ask the server to return the list of request methods it supports
- **TRACE**
  - Ask the server to return a diagnostic trace of the actions it takes
- **CONNECT**
  - A common extension method for Tunneling other protocols through HTTP
- **PUT, DELETE**
  - Used in HTTP publishing

24

Requesting AND sending data at the same time

- GET method:
  - The input is uploaded through the URL field of the request line:

    http://localhost/action-page.php?fname=Giovanna

  - Limited amount of data can be posted

- POST method:
  - The input is uploaded to server in message body
  - Unlimited amount of data can be posted

25

Idempotent methods

An HTTP method is **idempotent** if an identical request can be made once or several times in a row with the same effect while leaving the server in the same state.

An idempotent method should not have any side-effects

Implemented correctly, the GET, HEAD, PUT, and DELETE methods are **idempotent**, but not the POST method.

26

Idempotent methods

- GET /pageX HTTP/1.1 is idempotent. Called several times in a row, the client gets the same results:
  - GET /pageX HTTP/1.1
  - GET /pageX HTTP/1.1
  - GET /pageX HTTP/1.1
- POST /add_row HTTP/1.1 is not idempotent; if it is called several times, it adds several rows:
  - POST /add_row HTTP/1.1
  - POST /add_row HTTP/1.1 -> Adds a 2nd row
  - POST /add_row HTTP/1.1 -> Adds a 3rd row
- DELETE /idX/delete HTTP/1.1 is idempotent, even if the returned status code may change between requests:
  - DELETE /idX/delete HTTP/1.1 -> Returns 200 if idX exists
  - DELETE /idX/delete HTTP/1.1 -> Returns 404 as it just got deleted
  - DELETE /idX/delete HTTP/1.1 -> Returns 404

27

Safe methods

A HTTP method is safe if it doesn't alter the state of the server.

A method is safe if it leads to a read-only operation.

Several common HTTP methods are safe: GET and HEAD. All safe methods are also idempotent, but not all idempotent methods are safe.

For example, PUT and DELETE are both idempotent but unsafe.

What about POST?

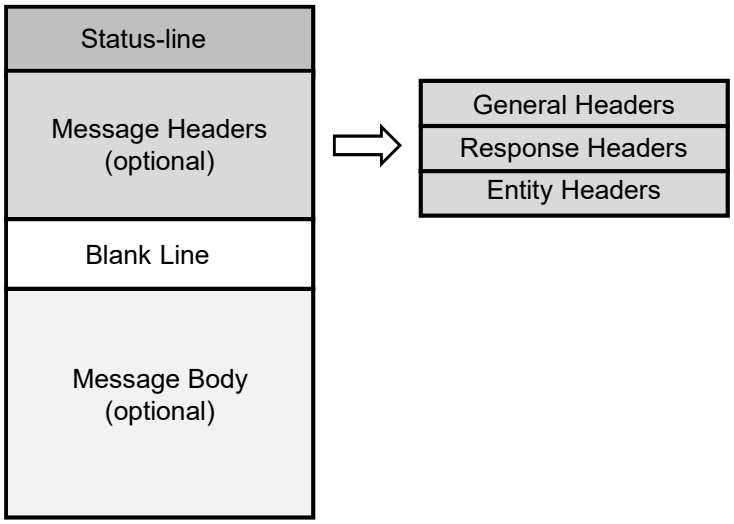
28

Q

How does the HTTP response composed ?

29

HTTP response format



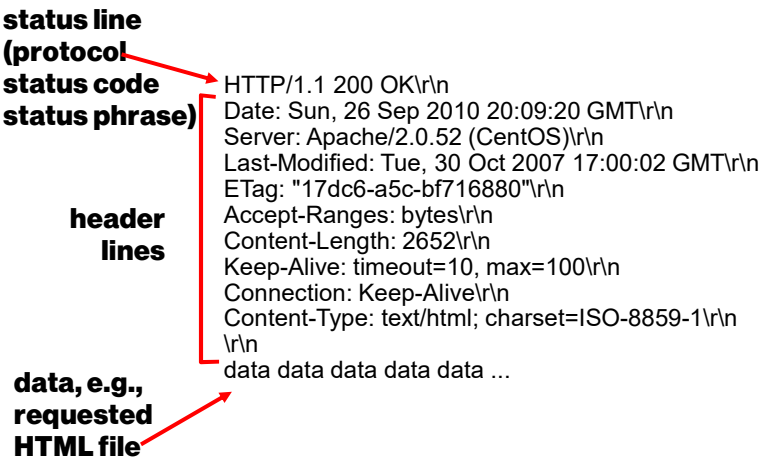
30

HTTP responses

- HTTP responses are a type of Internet Messages (RFC 822), and have the format:
  - A Start Line, followed by a CRLF
    Status Line
  - Zero or more Message Headers
    Field-name ":" [field-value] CRLF
  - An empty line
    Two CRLFs mark the end of the Headers
  - An optional Message Body if there is a payload
    All or part of the "Entity Body" or "Entity"

31

An example of a HTTP response message



32

A closer look at the Status Line

- Consists of three major parts:
  - HTTP Version
  - Status Code: one among 5 groups of 3 digit integers indicating the result of the attempt to satisfy the request
  - Status phrase: short textual description of the status code

Table 3.2 HTTP Status Code Categories

Status Code	Meaning
100-199	Informational: the server received the request but a final result is not yet available.
200-299	Success: the server was able to act on the request successfully.
300-399	Redirection: the client should redirect the request to a different server or resource.
400-499	Client error: the request contained an error that prevented the server from acting on it successfully.
500-599	Server error: the server failed to act on a request even though the request appears to be valid.

33

Status codes examples

- 200 OK
  - request succeeded, requested object later in this msg
- 301 Moved Permanently
  - requested object moved, new location specified later in this msg (Location:)
- 400 Bad Request
  - request msg not understood by server
- 404 Not Found
  - requested document not found on this server
- 505 HTTP Version Not Supported

34

A Closer Look at HTTP Headers

Headers come in four major types:

- **General Headers**
  - Provide info about messages of both kinds
- **Request Headers**
  - Provide request-specific info
- **Response Headers**
  - Provide response-specific info
- **Entity Headers**
  - Provide info about request and response entities

35

General Headers

- **Connection** – lets clients and servers manage connection state
  - Connection: Keep-Alive
  - Connection: close
- **Date** – when the message was created
  - Date: Sat, 31-May-03 15:00:00 GMT
- **Via** – shows proxies that handled message
  - Via: 1.1 www.myproxy.com (Squid/1.4)
- **Cache-Control** – Among the most complex of headers, enables caching directives
  - Cache-Control: no-cache

36

Request Headers

- **Host** – The hostname (and optionally port) of server to which request is being sent
- **Referer** – The URL of the resource from which the current request URI came
- **User-Agent** – Name of the requesting application, used in browser sensing
- **Accept** and its variants – Inform servers of client's capabilities and preferences
  - Enables content negotiation
  - Accept: image/gif, image/jpeg;q=0.5
  - Accept- variants for Language, Encoding, Charset
- **Cookie** How clients pass cookies back to the servers that set them
  - Cookie: id=23432;level=3

37

Response Headers

- **Server** – The server's name and version
  - Server: Microsoft-IIS/5.0
  - Can be problematic for security reasons
- **Set-Cookie** – This is how a server sets a cookie on a client
  - Set-Cookie: id=234; path=/shop; expires=Sat, 31-May-03 15:00:00 GMT; secure

38

Entity Headers

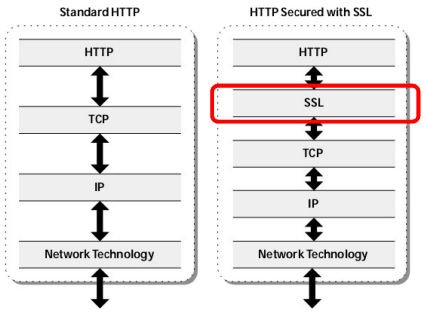
- **Allow** – Lists the request methods that can be used on the entity
  - Allow: GET, HEAD, POST
- **Location –** Gives the alternate or new location of the entity
  - Used with 3xx response codes (redirects)
  - Location: http://www.iugaza.edu.ps/ar/
- **Content-Encoding –** specifies encoding performed on the body of the response
  - Used with HTTP compression
  - Corresponds to Accept-Encoding request header
  - Content-Encoding: gzip
- **Content-Length –** The size of the entity body in bytes
- **Content-Type –** specifies Media (MIME) type of the entity body

39

Something about HTTPS (HTTP +SSL)

- (HTTPS) Hypertext Transfer Protocol over Secure Socket Layer (SSL)
- First implementation of HTTP over SSL was issued in 1995 by Netscape

Figure 4.10 ▶
The SSL protocol inserts itself between an application like HTTP and the TCP transport layer. TCP sees SSL as just another application, and HTTP communicates with SSL much the same as it does with TCP.



HTTPS is only slightly slower than HTTP

HTTPS is a bit more complex to set up (due to the need of a certificate)

40

Q

Quiz time!

41

Let's try!

- What does the server if a GET request contains an invalid query parameter?
- What is the difference between PUT and POST?
- How does the server respond if a POST request is sent without a request body?
- Let's suppose we want to bookmark a page, should I use GET or POST?

42